Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт

З ЛАБОРАТОРНОЇ РОБОТИ № 4

з дисципліни «Професійний Python»
на тему: «Професійне ООП та система типів Python»

Виконала:
студентка 3 курсу
групи ФеП-33
Наборська Дарія
Русланівна

Перевірив:
професор Свелеба Сергій
Андрійович

Львів-2026

Тема роботи

Лабораторна робота № 4. Професійне ООП та система типів Python.

Мета роботи

Сформувати практичні навички побудови професійної об'єктно-орієнтованої моделі в Python із використанням класів, композиції, наслідування, абстрактних базових класів, протоколів, dataclass, properties, dunder methods, принципів SOLID та сучасної системи type hints.

Номер індивідуального варіанта

Варіант 9. Система обліку спортивних результатів.

Опис предметної області

Система зберігає спортсменів, види спорту та змагання і реєструє результати спортсменів на змаганнях. Спортсмен належить до одного виду спорту і має спортивну категорію (МСМК, МС, КМС, розряди). Змагання проводиться з одного виду спорту, тому зареєструвати можна лише спортсмена цього виду і лише один раз. Результат оцінюється в балах від 0 до 100. Система дозволяє:

- реєструвати результат спортсмена (у тому числі з зовнішнього payload);
- шукати результати за видом спорту;
- визначати найкращий результат;
- обчислювати середній результат;
- формувати рейтинг за різними стратегіями (найкращий, середній, сума балів);
- зберігати об'єкти у generic repository;
- виводити рейтинг через reporting Protocol у різних форматах.

Завдання підвищеної складності

Для максимальної оцінки потрібно щонайменше три додаткові можливості. Реалізовано:

- **parameterized generic repository** — Repository[K, T] / InMemoryRepository[K, T];
- **generic service** — QueryService[E];
- **constrained TypeVar** — Number = TypeVar("Number", int, float) у average;
- **bound TypeVar** — T = TypeVar("T", bound=Identified), K bound=Hashable;
- **runtime_checkable Protocol** — Identified і Reporter перевіряються isinstance;
- **custom collection class** — RankingTable і Competition з __getitem__, __iter__, __contains__, __len__;

- **arithmetic dunder method і total_ordering** — Score.__add__, __radd__, __lt__ + @total_ordering;
- **strategy pattern** — BestResultStrategy, AverageResultStrategy, TotalScoreStrategy;
- **repository abstraction, immutable value objects, custom exception hierarchy**;
- **interface segregation через декілька Protocols** — Identified, Reporter, Exporter;
- **dependency injection container вручну** — Container.

Репозиторій: https://github.com/dashanaborska/python_lab, каталог Lab_04 (звіт — у Lab_04/report/).

Список domain entities

Клас	Вид	Відповідальність
Athlete	class	спортсмен: ID, ім'я з валідацією, вид спорту, категорія
Sport	frozen dataclass	вид спорту та одиниця вимірювання
Competition	dataclass	змагання; містить і реєструє результати
Result	frozen dataclass	результат спортсмена на змаганні
RankingTable	class (collection)	рейтингова таблиця з RankingEntry
RankingEntry	frozen dataclass	рядок рейтингу: місце, спортсмен, значення
Score	frozen value object	бали 0–100 з валідацією і порівнянням
AthleteId	frozen value object	додатний ідентифікатор спортсмена
Category	StrEnum	спортивна категорія
RankingStrategy	ABC	базова стратегія рейтингу
Reporter, Exporter, Identified	Protocol	абстракції звітів, експорту, ідентифікації
Repository, InMemoryRepository	generic ABC / class	зберігання об'єктів за ключем
ResultPayload	TypedDict	зовнішній payload результату
ResultService	application service	сценарії обліку результатів

Container	dataclass	ручне збирання залежностей

Class diagram

```
None
                    RankingStrategy (ABC)
                    + build(results) -> RankingTable
                    + aggregate(scores) [abstract]
                                ▲
                                │
        ┌───────────────────────┼──────────────────────┐
BestResultStrategy   AverageResultStrategy   TotalScoreStrategy

Sport ◄─── Athlete ──────► Category (StrEnum)
  ▲          │  └────────► AthleteId (value object)
  │          │
Competition ◆─── Result ──► Score (value object, total_ordering)
  (has many Result)

RankingTable ◆─── RankingEntry ──► Athlete

ResultService
  ├── Repository[int, Athlete]       (ABC, Generic[K, T])
  ├── Repository[int, Competition]      ▲
  ├── RankingStrategy (ABC)             InMemoryRepository[K, T]
  └── Reporter (Protocol) ◄─ ConsoleReporter, MarkdownReporter
                Exporter (Protocol) ◄─ DictExporter
                Identified (Protocol) ◄─ Athlete, Competition

Container ──creates──► ResultService, QueryService[Athlete]
ResultPayload (TypedDict) ──mapper──► Athlete, Score
```

◆ — composition (об'єкт містить і керує життєвим циклом частин), ▲ — inheritance / реалізація абстракції, ──► — асоціація.

Пояснення inheritance

Наслідування використано там, де є справжнє відношення is-a: кожна стратегія рейтингу є RankingStrategy. Базовий клас реалізує спільний алгоритм build (групування за спортсменом, сортування, нумерація місць), а підкласи перевизначають лише aggregate — це template method. Підкласи можна підставляти замість базового класу без зміни ResultService (LSP). Також InMemoryRepository наслідує абстрактний Repository, а винятки утворюють ієрархію SportResultsError → ValidationError, NotFoundError, DuplicateError.

```python
from abc import ABC, abstractmethod
from collections import defaultdict
from collections.abc import Iterable, Sequence
from typing import TypeVar

from sport_results.models import Athlete, RankingEntry, RankingTable, Result
```

```
Number = TypeVar("Number", int, float)


def average(values: Sequence[Number]) -> float:
    if not values:
        return 0.0
    return sum(values) / len(values)


class RankingStrategy(ABC):
    title: str = "Рейтинг"

    def build(self, results: Iterable[Result]) -> RankingTable:
        grouped: defaultdict[Athlete, list[float]] = defaultdict(list)
        for result in results:
            grouped[result.athlete].append(result.score.value)
        values = {
            athlete: self.aggregate(scores)
            for athlete, scores in grouped.items()
        }
        ordered = sorted(
            values.items(), key=lambda item: (-item[1], item[0].name)
        )
        entries = [
            RankingEntry(place, athlete, round(value, 2))
            for place, (athlete, value) in enumerate(ordered, start=1)
        ]
        return RankingTable(self.title, entries)

    @abstractmethod
    def aggregate(self, scores: list[float]) -> float:
        raise NotImplementedError


class BestResultStrategy(RankingStrategy):
    title = "Рейтинг за найкращим результатом"

    def aggregate(self, scores: list[float]) -> float:
        return max(scores)


class AverageResultStrategy(RankingStrategy):
    title = "Рейтинг за середнім результатом"

    def aggregate(self, scores: list[float]) -> float:
        return average(scores)


class TotalScoreStrategy(RankingStrategy):
    title = "Рейтинг за сумою балів"

    def aggregate(self, scores: list[float]) -> float:
        return sum(scores)
```

Пояснення composition

Composition (has-a) — основний спосіб побудови моделі: Competition містить список Result, Result містить Athlete і Score, Athlete містить Sport, Category та

AthleteId, RankingTable містить RankingEntry, а ResultService складається з repositories, стратегії і reporter, отриманих ззовні. Зміна будь-якого компонента (наприклад, інший reporter) не вимагає змінювати інші класи.

Python

```
@dataclass
class Competition:
    id: int
    title: str
    sport: Sport
    held_on: date
    _results: list[Result] = field(default_factory=list, repr=False)

    def register(self, athlete: Athlete, score: Score) -> Result:
        if athlete.sport != self.sport:
            raise ValidationError(
                f"{athlete.name} не виступає у виді спорту {self.sport}"
            )
        if athlete in self:
            raise DuplicateError(f"{athlete.name} вже має результат")
        result = Result(athlete, score, self.title)
        self._results.append(result)
        return result

    def __len__(self) -> int:
        return len(self._results)

    def __iter__(self) -> Iterator[Result]:
        return iter(self._results)

    def __contains__(self, athlete: object) -> bool:
        return any(result.athlete == athlete for result in self._results)
```

Приклад encapsulation

Стан Athlete зберігається у «захищених» атрибутах _id, _name, _sport, _category. Змінювати їх напряму не передбачено: ID і вид спорту доступні лише для читання, ім'я змінюється тільки через setter із валідацією, категорія — через метод promote.

Python

```
class Athlete:
    def __init__(
        self,
        athlete_id: AthleteId,
        name: str,
        sport: Sport,
        category: Category,
    ) -> None:
        self._id = athlete_id
        self._name = ""
        self.name = name
        self._sport = sport
        self._category = category

    @property
```

```
    def id(self) -> AthleteId:
        return self._id

    @property
    def name(self) -> str:
        return self._name

    @name.setter
    def name(self, value: str) -> None:
        cleaned = " ".join(value.split())
        if not cleaned or any(char.isdigit() for char in cleaned):
            raise ValidationError(f"Некоректне ім'я спортсмена: {value!r}")
        self._name = cleaned

    @property
    def sport(self) -> Sport:
        return self._sport

    @property
    def category(self) -> Category:
        return self._category

    def promote(self, category: Category) -> None:
        self._category = category

    def __repr__(self) -> str:
        return f"Athlete(id={self._id}, name={self._name!r})"

    def __eq__(self, other: object) -> bool:
        return isinstance(other, Athlete) and self._id == other._id

    def __hash__(self) -> int:
        return hash(self._id)
```

Приклад property

```
Python
@property
def name(self) -> str:
    return self._name

@name.setter
def name(self, value: str) -> None:
    cleaned = " ".join(value.split())
    if not cleaned or any(char.isdigit() for char in cleaned):
        raise ValidationError(f"Некоректне ім'я спортсмена: {value!r}")
    self._name = cleaned
```

Property перевіряє, що ім'я не порожнє й не містить цифр, і нормалізує пробіли. Зовнішній код працює з athlete.name як з полем, але некоректний стан створити неможливо (рисунок 3, остання помилка).

Приклад dataclass

```
Python
@dataclass(frozen=True, slots=True)
```

```
class Result:
    athlete: Athlete
    score: Score
    competition: str

    def __str__(self) -> str:
        return f"{self.athlete.name}: {self.score}"


@dataclass(frozen=True, slots=True)
class RankingEntry:
    place: int
    athlete: Athlete
    value: float


@dataclass(frozen=True, slots=True)
class Sport:
    name: str
    unit: str = "бали"

    def __str__(self) -> str:
        return self.name
```

Dataclass автоматично генерує __init__, __repr__, __eq__ (а для frozen=True — і __hash__). У Competition для списку результатів використано field(default_factory=list), щоб кожне змагання мало власний список.

Приклад immutable value object

Python

```
from dataclasses import dataclass
from enum import StrEnum
from functools import total_ordering

from sport_results.exceptions import ValidationError


class Category(StrEnum):
    MSMK = "МСМК"
    MS = "МС"
    KMS = "КМС"
    FIRST = "I розряд"
    SECOND = "II розряд"
    THIRD = "III розряд"


@total_ordering
@dataclass(frozen=True, slots=True)
class Score:
    value: float

    MIN = 0.0
    MAX = 100.0

    def __post_init__(self) -> None:
        if not self.MIN <= self.value <= self.MAX:
            raise ValidationError(
                f"Результат {self.value} поза межами [{self.MIN}; {self.MAX}]"
```

```
            )

    def __add__(self, other: "Score") -> float:
        return self.value + other.value

    def __radd__(self, other: float) -> float:
        return other + self.value

    def __lt__(self, other: object) -> bool:
        if not isinstance(other, Score):
            return NotImplemented
        return self.value < other.value

    def __str__(self) -> str:
        return f"{self.value:.2f}"


@dataclass(frozen=True, slots=True)
class AthleteId:
    value: int

    def __post_init__(self) -> None:
        if self.value <= 0:
            raise ValidationError("ID спортсмена має бути додатним")

    def __str__(self) -> str:
        return f"A-{self.value:03d}"
```

Score і AthleteId — frozen value objects: після створення їх не можна змінити (спроба присвоєння дає FrozenInstanceError), вони порівнюються за значенням, а __post_init__ гарантує коректність (бали 0–100, додатний ID). slots=True зменшує пам'ять і забороняє нові атрибути.

Приклад ABC

Python

```
class Repository(ABC, Generic[K, T]):
    @abstractmethod
    def add(self, item: T) -> None:
        raise NotImplementedError

    @abstractmethod
    def get(self, key: K) -> T:
        raise NotImplementedError

    @abstractmethod
    def all(self) -> list[T]:
        raise NotImplementedError
```

Repository — абстрактний generic клас: створити його екземпляр неможливо, а будь-яка реалізація зобов'язана визначити add, get, all. RankingStrategy також є ABC з abstractmethod aggregate.

Приклад Protocol

Python

```
from typing import Protocol, runtime_checkable

from sport_results.models import RankingTable
from sport_results.value_objects import AthleteId


@runtime_checkable
class Identified(Protocol):
    @property
    def id(self) -> AthleteId | int: ...


@runtime_checkable
class Reporter(Protocol):
    def report(self, table: RankingTable) -> str: ...


class Exporter(Protocol):
    def export(self, table: RankingTable) -> list[dict[str, str]]: ...
```

Python

```
from sport_results.models import RankingTable


class ConsoleReporter:
    def report(self, table: RankingTable) -> str:
        lines = [table.title, "-" * 44]
        for entry in table:
            lines.append(
                f"{entry.place:>2}. {entry.athlete.name:18}"
                f"{entry.athlete.category:11}{entry.value:7.2f}"
            )
        return "\n".join(lines)


class MarkdownReporter:
    def report(self, table: RankingTable) -> str:
        lines = [
            f"### {table.title}",
            "| # | Спортсмен | Бали |",
            "|---|---|---|",
        ]
        for entry in table:
            lines.append(
                f"| {entry.place} | {entry.athlete.name} | {entry.value:.2f}
|"
            )
        return "\n".join(lines)


class DictExporter:
    def export(self, table: RankingTable) -> list[dict[str, str]]:
        return [
            {
                "place": str(entry.place),
                "athlete": entry.athlete.name,
                "value": f"{entry.value:.2f}",
            }
            for entry in table
        ]
```

ConsoleReporter і MarkdownReporter не наслідують Reporter, але задовольняють його структурно (structural typing): достатньо мати метод report(table) -> str. Завдяки @runtime_checkable можна виконати isinstance(obj, Reporter). Інтерфейси розділено на дрібні Protocols (Reporter, Exporter, Identified) — ISP.

Dunder methods

Метод	Клас	Призначення
__init__	Athlete, RankingTable, InMemoryRepository	ініціалізація стану
__repr__	Athlete, RankingTable	зручне представлення для налагодження
__str__	Score, AthleteId, Sport, Result	текст для користувача
__eq__ / __hash__	Athlete	рівність за ID, використання як ключа dict
__lt__ + total_ordering	Score	порівняння <, <=, >, >= та max()
__add__ / __radd__	Score	Score + Score, sum(scores, 0.0)
__len__	Competition, RankingTable, InMemoryRepository	len()
__iter__	Competition, RankingTable, InMemoryRepository	for, list()
__contains__	Competition, RankingTable, InMemoryRepository	оператор in
__getitem__	RankingTable	table[1] — запис за місцем
__post_init__	Score, AthleteId	валідація dataclass

Python

```
class RankingTable:
    def __init__(self, title: str, entries: list[RankingEntry]) -> None:
        self.title = title
        self._entries = list(entries)

    def __len__(self) -> int:
        return len(self._entries)

    def __iter__(self) -> Iterator[RankingEntry]:
        return iter(self._entries)

    def __getitem__(self, place: int) -> RankingEntry:
        if not 1 <= place <= len(self._entries):
            raise IndexError(f"Немає місця {place}")
        return self._entries[place - 1]
```

```
    def __contains__(self, athlete: object) -> bool:
        return any(entry.athlete == athlete for entry in self._entries)

    def __repr__(self) -> str:
        return f"RankingTable({self.title!r}, entries={len(self)})"
```

Generic class

```python
class InMemoryRepository(Repository[K, T]):
    def __init__(self, key: Callable[[T], K]) -> None:
        self._key = key
        self._items: dict[K, T] = {}

    def add(self, item: T) -> None:
        key = self._key(item)
        if key in self._items:
            raise DuplicateError(f"Об'єкт з ключем {key} вже існує")
        self._items[key] = item

    def get(self, key: K) -> T:
        try:
            return self._items[key]
        except KeyError:
            raise NotFoundError(f"Об'єкт {key} не знайдено") from None

    def all(self) -> list[T]:
        return list(self._items.values())

    def find(self, predicate: Callable[[T], bool]) -> list[T]:
        return [item for item in self._items.values() if predicate(item)]

    def __len__(self) -> int:
        return len(self._items)

    def __iter__(self) -> Iterator[T]:
        return iter(self._items.values())

    def __contains__(self, key: object) -> bool:
        return key in self._items


def athlete_repository() -> InMemoryRepository[int, Athlete]:
    return InMemoryRepository(lambda athlete: athlete.id.value)


def competition_repository() -> InMemoryRepository[int, Competition]:
    return InMemoryRepository(lambda competition: competition.id)


class QueryService(Generic[E]):
    def __init__(self, repository: Repository[int, E]) -> None:
        self._repository = repository

    def all(self) -> list[E]:
        return self._repository.all()

    def count(self) -> int:
        return len(self._repository.all())
```

InMemoryRepository[K, T] параметризується типом ключа і типом сутності. Для athlete_repository mypy знає, що get повертає Athlete, а add приймає лише Athlete, — саме це виявило помилку №3 у type-checking experiment.

TypeVar

```Python
K = TypeVar("K", bound=Hashable)
T = TypeVar("T", bound=Identified)
E = TypeVar("E", bound=Identified)
Number = TypeVar("Number", int, float)


def average(values: Sequence[Number]) -> float:
    if not values:
        return 0.0
    return sum(values) / len(values)
```

K обмежений Hashable (ключ dict), T і E — bound TypeVar: сутність повинна мати property id (Protocol Identified). Number — constrained TypeVar: average приймає послідовність лише int або лише float.

TypedDict

```Python
from typing import NotRequired, TypedDict

from sport_results.exceptions import ValidationError
from sport_results.models import Athlete, Sport
from sport_results.value_objects import AthleteId, Category, Score


class ResultPayload(TypedDict):
    athlete_id: int
    athlete_name: str
    sport: str
    category: str
    competition_id: int
    score: float
    comment: NotRequired[str]


def athlete_from_payload(payload: ResultPayload) -> Athlete:
    try:
        category = Category(payload["category"])
    except ValueError:
        raise ValidationError(
            f"Невідома категорія {payload['category']!r}"
        ) from None
    return Athlete(
        AthleteId(payload["athlete_id"]),
        payload["athlete_name"],
        Sport(payload["sport"]),
        category,
    )
```

```
def score_from_payload(payload: ResultPayload) -> Score:
    return Score(float(payload["score"]))
```

ResultPayload описує зовнішні дані (наприклад, JSON із форми чи API): це звичайний dict з відомими ключами, NotRequired-поле comment необов'язкове. TypedDict не є domain entity — він не має поведінки та інваріантів, тому функції-mapper перетворюють payload у Athlete і Score з повною валідацією.

Application service

Python

```
from collections.abc import Iterator
from typing import Generic, TypeVar

from sport_results.dto import (
    ResultPayload,
    athlete_from_payload,
    score_from_payload,
)
from sport_results.exceptions import NotFoundError
from sport_results.models import Athlete, Competition, RankingTable, Result
from sport_results.protocols import Identified, Reporter
from sport_results.repositories import Repository
from sport_results.strategies import RankingStrategy, average
from sport_results.value_objects import Score

E = TypeVar("E", bound=Identified)


class QueryService(Generic[E]):
    def __init__(self, repository: Repository[int, E]) -> None:
        self._repository = repository

    def all(self) -> list[E]:
        return self._repository.all()

    def count(self) -> int:
        return len(self._repository.all())


class ResultService:
    def __init__(
        self,
        athletes: Repository[int, Athlete],
        competitions: Repository[int, Competition],
        strategy: RankingStrategy,
        reporter: Reporter,
    ) -> None:
        self._athletes = athletes
        self._competitions = competitions
        self._strategy = strategy
        self._reporter = reporter

    def register_athlete(self, athlete: Athlete) -> None:
        self._athletes.add(athlete)
```

```
    def register_result(
        self,
        competition_id: int,
        athlete_id: int,
        score: float,
    ) -> Result:
        competition = self._competitions.get(competition_id)
        athlete = self._athletes.get(athlete_id)
        return competition.register(athlete, Score(score))

    def register_payload(self, payload: ResultPayload) -> Result:
        athlete = athlete_from_payload(payload)
        try:
            athlete = self._athletes.get(athlete.id.value)
        except NotFoundError:
            self._athletes.add(athlete)
        competition = self._competitions.get(payload["competition_id"])
        return competition.register(athlete, score_from_payload(payload))

    def results(self, sport: str | None = None) -> Iterator[Result]:
        for competition in self._competitions.all():
            if sport is None or competition.sport.name == sport:
                yield from competition

    def find_by_sport(self, sport: str) -> list[Result]:
        return list(self.results(sport))

    def best_result(self, sport: str | None = None) -> Result:
        results = list(self.results(sport))
        if not results:
            raise NotFoundError("Результатів немає")
        return max(results, key=lambda result: result.score)

    def average_result(self, sport: str | None = None) -> float:
        return average([result.score.value for result in self.results(sport)])

    def ranking(self, sport: str | None = None) -> RankingTable:
        return self._strategy.build(self.results(sport))

    def use_strategy(self, strategy: RankingStrategy) -> None:
        self._strategy = strategy

    def report(self, sport: str | None = None) -> str:
        return self._reporter.report(self.ranking(sport))
```

ResultService реалізує сценарії: реєстрацію спортсмена і результату, реєстрацію з payload, пошук за видом спорту, найкращий і середній результат, рейтинг і звіт. Він не знає, як зберігаються дані та як друкується звіт, — працює лише з абстракціями.

Приклад dependency injection

Python

```
from dataclasses import dataclass, field
from datetime import date

from sport_results.models import Athlete, Competition, Sport
from sport_results.protocols import Reporter
```

```
from sport_results.reporters import ConsoleReporter
from sport_results.repositories import (
    InMemoryRepository,
    athlete_repository,
    competition_repository,
)
from sport_results.services import QueryService, ResultService
from sport_results.strategies import BestResultStrategy, RankingStrategy
from sport_results.value_objects import AthleteId, Category


@dataclass
class Container:
    strategy: RankingStrategy = field(default_factory=BestResultStrategy)
    reporter: Reporter = field(default_factory=ConsoleReporter)
    athletes: InMemoryRepository[int, Athlete] = field(
        default_factory=athlete_repository
    )
    competitions: InMemoryRepository[int, Competition] = field(
        default_factory=competition_repository
    )

    def result_service(self) -> ResultService:
        return ResultService(
            self.athletes, self.competitions, self.strategy, self.reporter
        )

    def athlete_queries(self) -> QueryService[Athlete]:
        return QueryService(self.athletes)


SWIMMING = Sport("Плавання")
ATHLETICS = Sport("Легка атлетика")
BOXING = Sport("Бокс")


def seed(container: Container) -> None:
    athletes = [
        Athlete(AthleteId(1), "Олена Ковальчук", SWIMMING, Category.KMS),
        Athlete(AthleteId(2), "Максим Бондар", ATHLETICS, Category.MS),
        Athlete(AthleteId(3), "Ірина Мельник", SWIMMING, Category.FIRST),
        Athlete(AthleteId(4), "Андрій Шевченко", BOXING, Category.KMS),
        Athlete(AthleteId(5), "Софія Ткачук", ATHLETICS, Category.MSMK),
        Athlete(AthleteId(6), "Марія Бойко", SWIMMING, Category.MS),
    ]
    for athlete in athletes:
        container.athletes.add(athlete)
    competitions = [
        Competition(1, "Кубок Львова", SWIMMING, date(2026, 3, 14)),
        Competition(2, "Чемпіонат області", SWIMMING, date(2026, 4, 18)),
        Competition(3, "Весняний старт", ATHLETICS, date(2026, 4, 2)),
        Competition(4, "Турнір пам'яті", BOXING, date(2026, 5, 9)),
    ]
    for competition in competitions:
        container.competitions.add(competition)
```

Усі залежності ResultService передаються через конструктор. Container збирає їх вручну і дозволяє підмінити будь-яку: у тестах

Container(reporter=FakeReporter()) замінює звітність тестовим двійником без зміни сервісу.

Експеримент: сервіс без DI проти DI

```python
class FixedClock:
    def now(self) -> str:
        return "2026-05-01 10:00"


class TightlyCoupledService:
    def __init__(self) -> None:
        self.clock = FixedClock()

    def stamp(self, text: str) -> str:
        return f"[{self.clock.now()}] {text}"


class Clock:
    def __init__(self, source: Callable[[], str]) -> None:
        self._source = source

    def now(self) -> str:
        return self._source()


class InjectedService:
    def __init__(self, clock: Clock) -> None:
        self.clock = clock

    def stamp(self, text: str) -> str:
        return f"[{self.clock.now()}] {text}"
```

- **Легше тестувати** варіант B: у тесті передаємо Clock(lambda: "TEST-TIME") і отримуємо передбачуваний результат; у варіанті A годинник створюється всередині й підмінити його неможливо без monkeypatch.
- **Легше замінювати** варіант B: інша реалізація передається ззовні без зміни коду сервісу.
- **DIP** відповідає варіант B: сервіс залежить від абстракції, а не від конкретного класу.
- **Coupling** — composition з abstraction знижує зв'язність: сервіс знає лише інтерфейс компонента, тому компоненти можна розвивати й тестувати незалежно.

Дослідження inheritance vs composition

```python
class BaseProcessor:
    def process(self, line: str) -> str:
        name, value = self.parse(line)
        self.validate(value)
        return self.format(name, value)

    def parse(self, line: str) -> tuple[str, float]:
```

```
        name, value = line.split(",")
        return name.strip(), float(value)

    def validate(self, value: float) -> None:
        Score(value)

    def format(self, name: str, value: float) -> str:
        return f"{name}: {value:.1f}"


class CsvProcessor(BaseProcessor):
    def parse(self, line: str) -> tuple[str, float]:
        name, value = line.split(";")
        return name.strip(), float(value)


@dataclass
class Processor:
    parser: SemicolonParser
    validator: ScoreValidator
    formatter: UpperFormatter

    def process(self, line: str) -> str:
        name, value = self.parser.parse(line)
        self.validator.validate(value)
        return self.formatter.format(name, value)
```

Критерій	**Inheritance (CsvProcessor)**	**Composition (Processor)**
Coupling	сильний: підклас залежить від деталей базового класу	слабкий: лише інтерфейси parser/validator/formatter
Extensibility	нова комбінація = новий підклас	нова комбінація = інші компоненти в конструкторі
Testability	тестується вся ієрархія разом	кожен компонент тестується окремо, легко підставити заглушку
Повторне використання	лише через наслідування	ScoreValidator використовується будь-де

Пояснення SOLID principles

- **S — Single Responsibility:** Athlete відповідає за стан спортсмена, Competition — за реєстрацію результатів, стратегії — за агрегацію, reporters — за форматування, repository — за зберігання, ResultService — за сценарії.
- **O — Open/Closed:** новий спосіб рейтингу (TotalScoreStrategy) або новий формат звіту (MarkdownReporter) додається новим класом без зміни ResultService.
- **L — Liskov Substitution:** будь-яку стратегію можна підставити замість RankingStrategy через use_strategy, а будь-який Reporter — замість іншого; поведінка сервісу залишається коректною.

- **I — Interface Segregation:** замість одного великого інтерфейсу є дрібні Protocols Reporter, Exporter, Identified; клас реалізує лише те, що йому потрібно.
- **D — Dependency Inversion:** ResultService залежить від абстракцій Repository, RankingStrategy і Reporter, а конкретні реалізації передаються через Container.

Основний код program

Python

```
from sport_results.container import ATHLETICS, Container, seed
from sport_results.dto import ResultPayload
from sport_results.exceptions import SportResultsError
from sport_results.models import Athlete, Competition
from sport_results.protocols import Identified, Reporter
from sport_results.reporters import DictExporter, MarkdownReporter
from sport_results.strategies import AverageResultStrategy, TotalScoreStrategy
from sport_results.value_objects import AthleteId, Category, Score


def title(text: str) -> None:
    print(f"\n=== {text} ===")


def main() -> None:
    container = Container()
    seed(container)
    service = container.result_service()

    title("Реєстрація результатів")
    for competition_id, athlete_id, score in [
        (1, 1, 87.5), (1, 3, 78.4), (1, 6, 90.2),
        (2, 1, 91.0), (2, 3, 82.7), (2, 6, 88.1),
        (3, 2, 92.3), (3, 5, 95.1), (4, 4, 81.0),
    ]:
        result = service.register_result(competition_id, athlete_id, score)
        print(f"{result.competition:20}{result}")

    payload: ResultPayload = {
        "athlete_id": 7,
        "athlete_name": "Богдан Лисенко",
        "sport": "Легка атлетика",
        "category": "I розряд",
        "competition_id": 3,
        "score": 69.5,
    }
    print("TypedDict payload ->", service.register_payload(payload))

    title("Обробка помилок (custom exceptions)")
    for action in (
        lambda: service.register_result(1, 1, 99.0),
        lambda: service.register_result(3, 1, 80.0),
        lambda: service.register_result(1, 42, 80.0),
        lambda: Score(120),
        lambda: Athlete(AthleteId(9), "Тарас 1", ATHLETICS, Category.MS),
    ):
        try:
            action()
```

```
        except SportResultsError as error:
            print(f"{type(error).__name__}: {error}")

    title("Пошук за видом спорту: Плавання")
    for result in service.find_by_sport("Плавання"):
        print(f"{result.competition:20}{result}")
    best = service.best_result()
    print(f"Найкращий результат: {best} ({best.athlete.sport})")
    print(f"Середній результат: {service.average_result():.2f}")
    print(f"Середній (Плавання): {service.average_result('Плавання'):.2f}")

    title("Рейтинг (strategy + Reporter Protocol)")
    print(service.report("Плавання"))
    service.use_strategy(AverageResultStrategy())
    print(service.report("Плавання"))
    service.use_strategy(TotalScoreStrategy())
    table = service.ranking()
    print(service.report())

    title("Dunder methods")
    competition: Competition = container.competitions.get(1)
    olena = container.athletes.get(1)
    print("len(competition) =", len(competition))
    print("olena in competition:", olena in competition)
    print("table[1] =", table[1].athlete.name, table[1].value)
    print("repr:", repr(table), repr(olena))
    print("Score(87.5) < Score(90.2):", Score(87.5) < Score(90.2))
    print("Score(87.5) + Score(90.2) =", Score(87.5) + Score(90.2))
    print("sum(scores) =", sum([Score(10), Score(20), Score(30)], 0.0))

    title("Інші реалізації Protocol")
    markdown: Reporter = MarkdownReporter()
    print(markdown.report(service.ranking("Легка атлетика")))
    print("DictExporter:", DictExporter().export(table)[:1])
    print("isinstance(MarkdownReporter(), Reporter):",
          isinstance(markdown, Reporter))
    print("isinstance(olena, Identified):", isinstance(olena, Identified))
    queries = container.athlete_queries()
    print("Generic QueryService[Athlete].count():", queries.count())


if __name__ == "__main__":
    main()
```

Screenshot execution

```
● (.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_04> python -m pip install -e ".[dev]"
    Found existing installation: sport-results 0.4.0
    Uninstalling sport-results-0.4.0:
      Successfully uninstalled sport-results-0.4.0
  Successfully installed sport-results-0.4.0
```

Рисунок 1 — Встановлення пакета з dev-залежностями

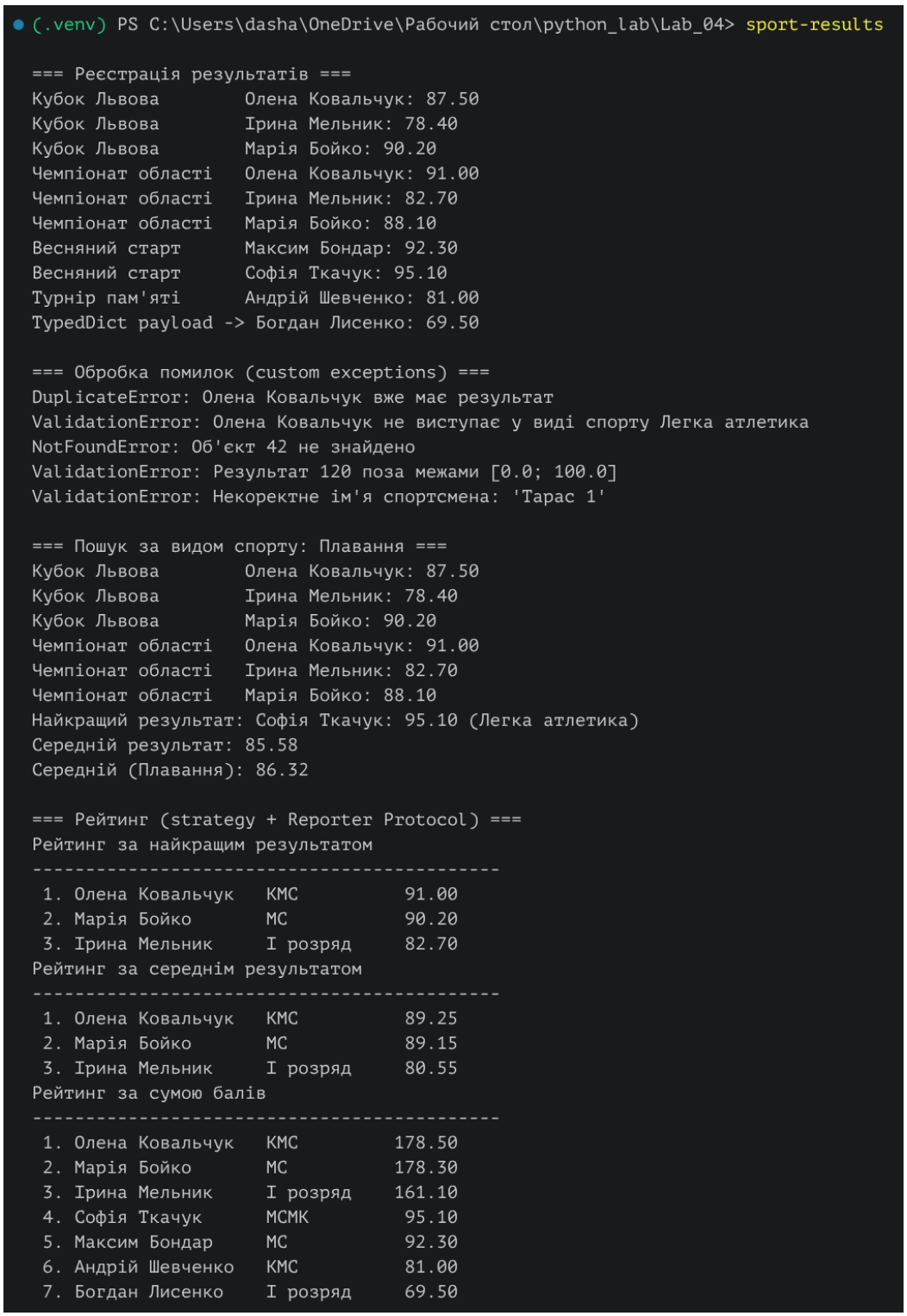


Рисунок 2 — Реєстрація результатів, обробка помилок, пошук, найкращий і середній результат, рейтинги

```
=== Dunder methods ===
len(competition) = 3
olena in competition: True
table[1] = Олена Ковальчук 178.5
repr: RankingTable('Рейтинг за сумою балів', entries=7) Athlete(id=A-001, name='Олена Ковальчук')
Score(87.5) < Score(90.2): True
Score(87.5) + Score(90.2) = 177.7
sum(scores) = 60.0

=== Інші реалізації Protocol ===
### Рейтинг за сумою балів
| # | Спортсмен | Бали |
|---|---|---|
| 1 | Софія Ткачук | 95.10 |
| 2 | Максим Бондар | 92.30 |
| 3 | Богдан Лисенко | 69.50 |
DictExporter: [{'place': '1', 'athlete': 'Олена Ковальчук', 'value': '178.50'}]
isinstance(MarkdownReporter(), Reporter): True
isinstance(olena, Identified): True
Generic QueryService[Athlete].count(): 7
```

Рисунок 3 — Dunder methods та інші реалізації Protocol

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_04> sport-results-design
Варіант A (сервіс сам створює залежність):
  [2026-05-01 10:00] результат збережено
Варіант B (dependency injection, у тесті підміняємо годинник):
  [TEST-TIME] результат збережено

Inheritance: CsvProcessor(BaseProcessor)
  Софія Ткачук: 95.1
Composition: Processor(parser, validator, formatter)
  СОФІЯ ТКАЧУК: 95.1
  валідатор спрацював: Результат 120.0 поза межами [0.0; 100.0]
```

Рисунок 4 — Експерименти DI та inheritance vs composition

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_04> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.13.16, pytest-9.1.1, pluggy-1.7.0 -- C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_04\.venv\Scripts\python.exe
rootdir: C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_04
configfile: pyproject.toml
collecting ... collected 10 items

tests/test_model.py::test_score_value_object PASSED                      [ 10%]
tests/test_model.py::test_athlete_encapsulation PASSED                    [ 20%]
tests/test_model.py::test_competition_composition_and_dunders PASSED      [ 30%]
tests/test_model.py::test_service_queries PASSED                          [ 40%]
tests/test_model.py::test_strategies_inheritance PASSED                   [ 50%]
tests/test_model.py::test_reporter_dependency_injection PASSED            [ 60%]
tests/test_model.py::test_payload_and_errors PASSED                       [ 70%]
tests/test_model.py::test_generic_repository PASSED                       [ 80%]
tests/test_model.py::test_constrained_average PASSED                      [ 90%]
tests/test_model.py::test_design_variants PASSED                          [100%]

============================== 10 passed in 0.05s ==============================
```

Рисунок 5 — Результат виконання тестів pytest

Результат mypy

Використано type checker mypy у режимі strict (налаштування в pyproject.toml, секція [tool.mypy]: strict = true, warn_unused_ignores, warn_return_any, mypy_path = "src"). Перевірка всього пакета:

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_04> mypy src
Success: no issues found in 13 source files
```

Рисунок 6 — mypy src: помилок немає

Приклади знайдених type errors

У файлі experiments/type_errors.py навмисно внесено три помилки:

```python
from datetime import date

from sport_results.models import Competition, Sport
from sport_results.repositories import athlete_repository

athlete_id: int = "1"


def total() -> int:
    return "100"


repository = athlete_repository()
repository.add(Competition(1, "Кубок", Sport("Плавання"), date(2026, 3, 14)))
```



Рисунок 7 — Diagnostics mypy для навмисних помилок

№	Помилка	Diagnostics mypy	Виправлення
1	athlete_id: int = "1"	Incompatible types in assignment [assignment]	athlete_id: int = 1
2	total() -> int повертає "100"	Incompatible return value type [return-value]	return 100
3	Competition у repository для Athlete	Argument 1 to "add" ... expected "Athlete" [arg-type]	окремі athlete_repository і competition_repository

Python виконав би цей код без помилок (type hints не перевіряються в runtime), а репозиторій мовчки зберіг би об'єкт не того типу. Static type checker знаходить проблему ще до запуску.

Виправлений результат type checking

```python
from datetime import date

from sport_results.models import Athlete, Competition, Sport
from sport_results.repositories import (
    athlete_repository,
    competition_repository,
)
```

```
from sport_results.value_objects import AthleteId, Category

athlete_id: int = 1


def total() -> int:
    return 100


swimming = Sport("Плавання")
competitions = competition_repository()
competitions.add(Competition(1, "Кубок", swimming, date(2026, 3, 14)))
athletes = athlete_repository()
athletes.add(Athlete(AthleteId(athlete_id), "Олена", swimming, Category.KMS))
```

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_04> mypy experiments/type_errors_fixed.py
Success: no issues found in 1 source file
```

Рисунок 8 — туру після виправлення

Результат git status та git log

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_04> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Рисунок 9 — Результат git status

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_04> git log --oneline -12
6702ce3 (HEAD -> main, origin/main, origin/HEAD) Add README for Lab_04
a010593 Add tests for object model
0dea18b Add type checking experiment
207b560 Add demo scenario and design experiments
477056b Add generic repository, DTO and application service
669de92 Add protocols, ranking strategies and reporters
86655a9 Add domain models
ee328ef Add Lab_04 structure, exceptions and value objects
e589ea8 Add Lab_03 report
7d0216d Improve pipeline stage log output
7570124 Add README for Lab_03
7acca20 Add tests for streaming pipeline
```

Рисунок 10 — Результат git log --oneline (останні 12 комітів)

Відповіді на контрольні запитання

1. Що таке class?

Шаблон (тип), що описує дані (атрибути) та поведінку (методи) об'єктів.

2. Що таке object?

Конкретний екземпляр класу зі своїм станом; у Python усе є об'єктом.

3. Що таке instance?

Об'єкт, створений із певного класу, наприклад Athlete(...) — instance класу Athlete.

4. Для чого __init__()?

Для ініціалізації стану нового об'єкта після його створення.

5. Що таке instance attribute?

Атрибут, що належить конкретному об'єкту (self._name), у кожного об'єкта свій.

6. Що таке class attribute?

Атрибут, визначений у класі й спільний для всіх екземплярів (RankingStrategy.title, Score.MIN).

7. Що таке encapsulation?

Приховування внутрішнього стану й надання контрольованого доступу через методи та properties.

8. Для чого properties?

Щоб надати доступ до атрибута як до поля, але з контролем: валідацією, обчисленням, read-only.

9. Чим property відрізняється від public field?

Public field можна змінити будь-яким значенням, property виконує код при читанні/записі і може заборонити запис.

10. Що таке inheritance?

Механізм створення класу на основі іншого з успадкуванням і можливістю перевизначення поведінки.

11. Що означає relation is-a?

Підклас є різновидом базового класу: BestResultStrategy is a RankingStrategy.

12. Що таке composition?

Побудова об'єкта з інших об'єктів, які він містить і використовує.

13. Що означає relation has-a?

Об'єкт містить інший об'єкт: Competition has Result, ResultService has Reporter.

14. Коли composition краща за inheritance?

Коли немає справжнього is-a, коли треба комбінувати або підмінювати поведінку, зменшувати coupling і спрощувати тестування.

15. Для чого super()?

Для виклику методів базового класу (наприклад, його __init__) з підкласу з урахуванням MRO.

16. Що таке polymorphism?

Можливість працювати з об'єктами різних класів через спільний інтерфейс: service.report() з будь-яким Reporter.

17. Що таке duck typing?

Підхід, коли важливі наявні методи об'єкта, а не його клас: «якщо крякає як качка — це качка».

18. Що таке ABC?

Abstract Base Class — клас з модуля abc, що задає обов'язковий інтерфейс для підкласів.

19. Для чого abstractmethod?

Позначає метод, який підклас обов'язково має реалізувати; без цього не можна створити instance.

20. Чи можна створити instance abstract class?

Ні, якщо в ньому є нереалізовані abstractmethod — виникає TypeError (перевірено тестом).

21. Що таке Protocol?

Опис структурного інтерфейсу з typing: тип підходить, якщо має потрібні методи/атрибути.

22. Чим Protocol відрізняється від ABC?

ABC вимагає явного наслідування (nominal typing), Protocol — лише відповідності структури (structural typing).

23. Що таке structural typing?

Сумісність типів визначається набором методів і атрибутів, а не ієрархією класів.

24. Що таке nominal typing?

Сумісність визначається явним оголошенням: клас має наслідувати потрібний тип.

25. Що таке dataclass?

Декоратор, що автоматично генерує службові методи для класів, які переважно зберігають дані.

26. Які methods генерує dataclass?

__init__, __repr__, __eq__; за параметрами — __lt__ та інші (order=True), __hash__ (frozen=True/unsafe_hash), __slots__ (slots=True).

27. Для чого default_factory?

Для створення нового значення за замовчуванням для кожного об'єкта (наприклад, порожнього list).

28. Чому mutable default є проблемою?

Один і той самий змінний об'єкт ділився б між усіма екземплярами; dataclass це забороняє.

29. Для чого __post_init__()?

Для додаткової ініціалізації та валідації після згенерованого __init__ (Score перевіряє межі).

30. Що означає frozen=True?

Екземпляр незмінний: присвоєння атрибутів піднімає FrozenInstanceError; об'єкт стає hashable.

31. Що означає slots=True?

Клас використовує __slots__: менше пам'яті, швидший доступ, неможливо додати нові атрибути.

32. Що таке dunder method?

Спеціальний метод з подвійним підкресленням (__len__, __add__), який Python викликає для вбудованих операцій.

33. Для чого __repr__()?

Для однозначного представлення об'єкта для розробника (налагодження, консоль).

34. Для чого __str__()?

Для зручного текстового представлення для користувача (print, str()).

35. Для чого __len__()?

Щоб функція len() працювала з об'єктом.

36. Для чого __iter__()?

Щоб об'єкт можна було перебирати у for і передавати у list(), sum() тощо.

37. Для чого __contains__()?

Щоб працював оператор in (athlete in competition).

38. Для чого __call__()?

Щоб екземпляр можна було викликати як функцію: obj().

39. Що таке SOLID?

П'ять принципів об'єктного дизайну: SRP, OCP, LSP, ISP, DIP.

40. Що означає SRP?

Single Responsibility — клас має одну причину для змін.

41. Що означає OCP?
Open/Closed — код відкритий для розширення, закритий для модифікації.

42. Що означає LSP?
Liskov Substitution — підклас можна підставити замість базового класу без порушення коректності.

43. Що означає ISP?
Interface Segregation — краще кілька вузьких інтерфейсів, ніж один великий.

44. Що означає DIP?
Dependency Inversion — модулі залежать від абстракцій, а не від конкретних реалізацій.

45. Що таке dependency injection?
Передача залежностей об'єкту ззовні (через конструктор), замість створення їх усередині.

46. Чому DI покращує testability?
У тестах можна передати fake або mock-реалізацію і перевіряти клас ізольовано.

47. Що таке type hint?
Анотація очікуваного типу змінної, параметра чи значення, що повертається.

48. Чи перевіряє Python type hints автоматично в runtime?
Ні, інтерпретатор їх не перевіряє; для цього потрібні static type checkers або окремі бібліотеки.

49. Що таке static type checker?
Інструмент, що аналізує код без запуску і знаходить невідповідності типів.

50. Для чого mypy?
Найпоширеніший static type checker для Python; використаний у цій роботі в strict-режимі.

51. Для чого Pyright?
Швидкий static type checker від Microsoft, основа Pylance у VS Code.

52. Що таке TypeVar?
Змінна типу для опису узагальнених функцій і класів.

53. Що таке Generic?
Базовий клас для створення параметризованих (узагальнених) класів.

54. Що означає Generic[T]?

Клас параметризується типом T, який підставляється при використанні: Repository[int, Athlete].

55. Що таке bound TypeVar?

TypeVar, що може бути лише вказаним типом або його підтипом (bound=Identified).

56. Що таке constrained TypeVar?

TypeVar, що може бути лише одним із переліку типів (TypeVar("Number", int, float)).

57. Що таке TypedDict?

Тип для dict з фіксованим набором ключів і типів значень.

58. Чим TypedDict відрізняється від dataclass?

TypedDict у runtime — звичайний dict без методів і валідації; dataclass — повноцінний клас з атрибутами, методами й можливістю валідації.

59. Коли доцільно використовувати TypedDict?

Для опису JSON, payload API, конфігурацій — даних, що приходять ззовні у вигляді dict.

60. Чому external payload і domain object краще розділяти?

Зовнішній формат може змінюватися і містити некоректні дані; mapper ізолює domain model, валідує дані і не дає зовнішнім змінам ламати бізнес-логіку.

Висновок

У лабораторній роботі побудовано типізовану об'єктну модель обліку спортивних результатів. Domain entities Athlete, Sport, Competition, Result і RankingTable реалізовано класами та dataclass, Score і AthleteId — незмінними value objects. Encapsulation і properties захищають стан спортсмена, composition використано як основний спосіб побудови моделі, inheritance — для стратегій рейтингу (ABC RankingStrategy) і generic repository. Protocols Reporter, Exporter, Identified задають структурні інтерфейси, dunder methods інтегрують класи з операторами Python. ResultService отримує всі залежності через dependency injection, що реалізує принципи SOLID і спрощує тестування (10 тестів pytest). Generic, TypeVar (bound і constrained) та TypedDict забезпечують точні типи, а mypy у strict-режимі не знаходить помилок у пакеті й виявив усі три навмисно внесені type errors.